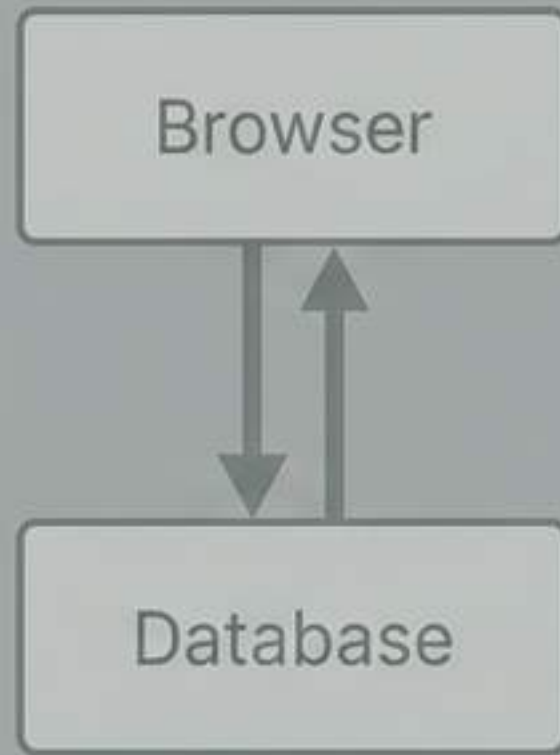


Operational Orchestration GUI

High Level Design & Architecture Blueprint

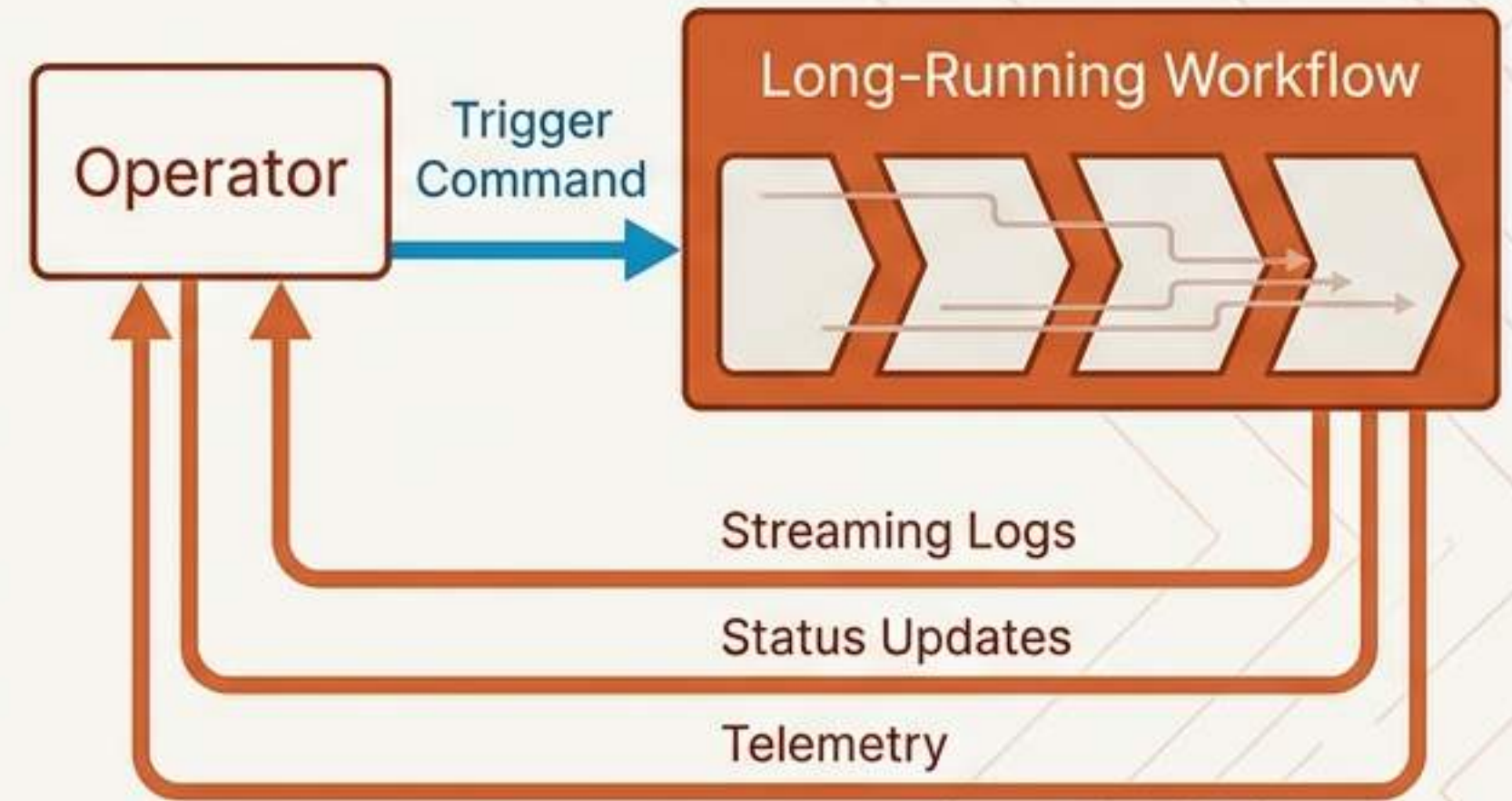


Traditional Web App (CRUD)



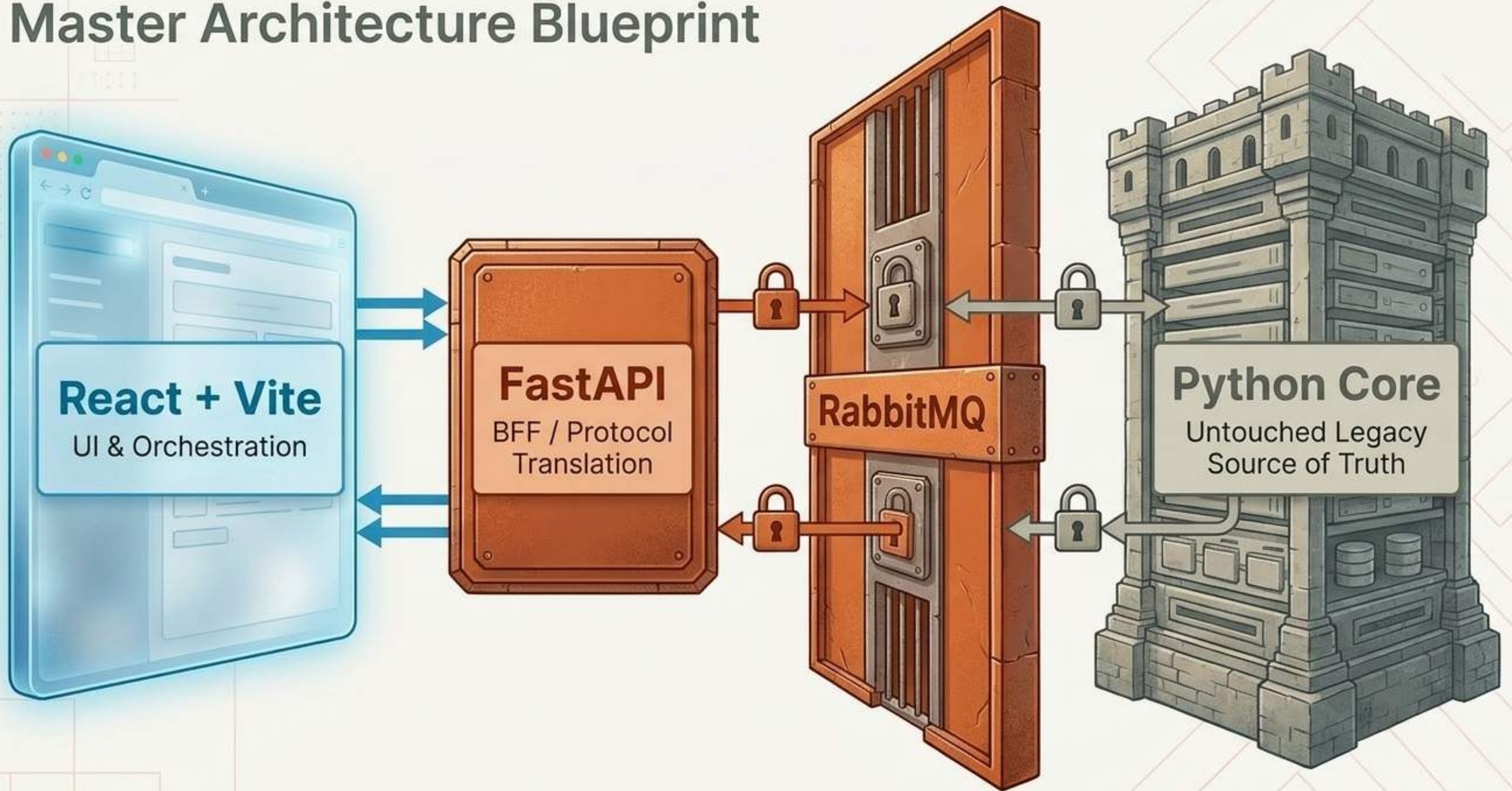
- Instant responses.
- Frontend logic.
- Synchronous data.

Async Orchestration Engine



- Backend-driven state.
- Thin UI orchestration layer.
- Zero legacy disruption.

Master Architecture Blueprint



Frontend Framework Selection Diagnostic

The Problem

Choose a frontend framework for a data-dense, internal operational GUI with no SEO requirements.

Pros & Cons

Angular	Pro: Enterprise structure.	Con: Too heavy, steep learning curve.
Next.js	Pro: Powerful SSR.	Con: Unnecessary complexity (No SSR needed).
React + Vite	Pro: Simple, massive ecosystem.	Con: Requires manual library assembly.

Alternatives



Angular



Next.js



React + Vite

Final Decision

React + Vite

Maximum delivery speed, simple mental model, and ideal for single-page operational tooling.

BFF Translation Layer

We require a Backend-For-Frontend to translate web requests into RabbitMQ messages.

- Node.js (Next Server)
- Python (FastAPI)

Decision Matrix & Architecture Logic

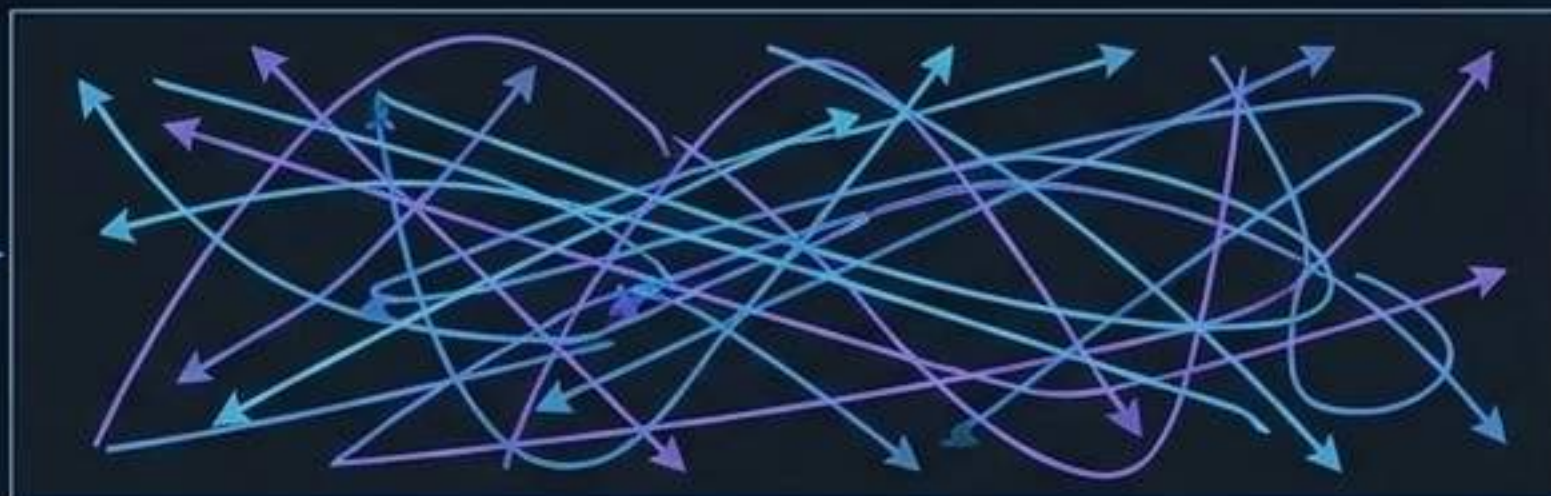
Node.js	FastAPI
Unified JS stack, but introduces context switching from our existing Python infrastructure.	Native async, flawless RabbitMQ integration, and shares models directly with our core.



Real-time Telemetry Strategy

How to handle UI commands & streaming logs?

WebSockets
(Bi-directional)



Pro: True real-time.

Con: High operational complexity, state-sync risks, reconnect bugs.

REST + SSE
(Uni-directional)

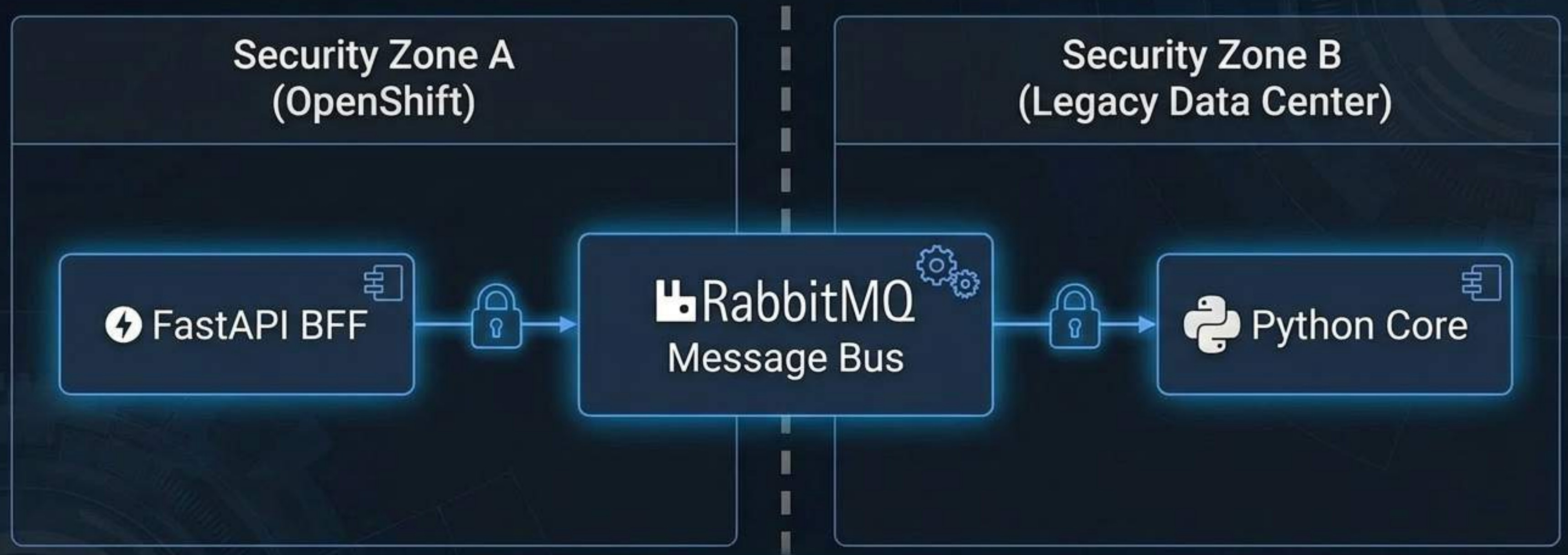


Pro: Matches asymmetric traffic, automatic reconnects, highly stable.

Con: One-way only.

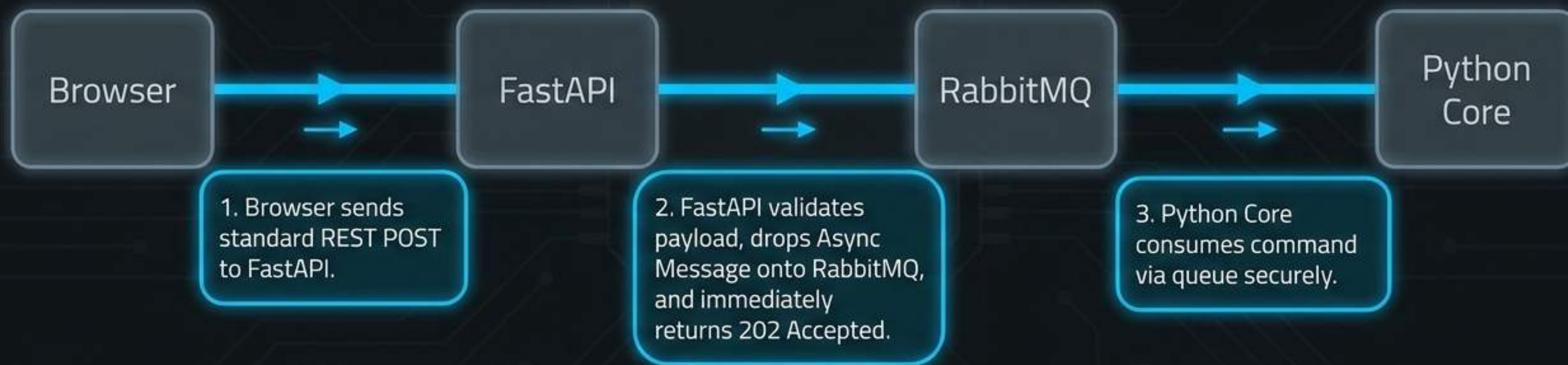
Final Decision: REST for Commands. SSE (with Polling MVP) for Events.

Deployment Network Independence

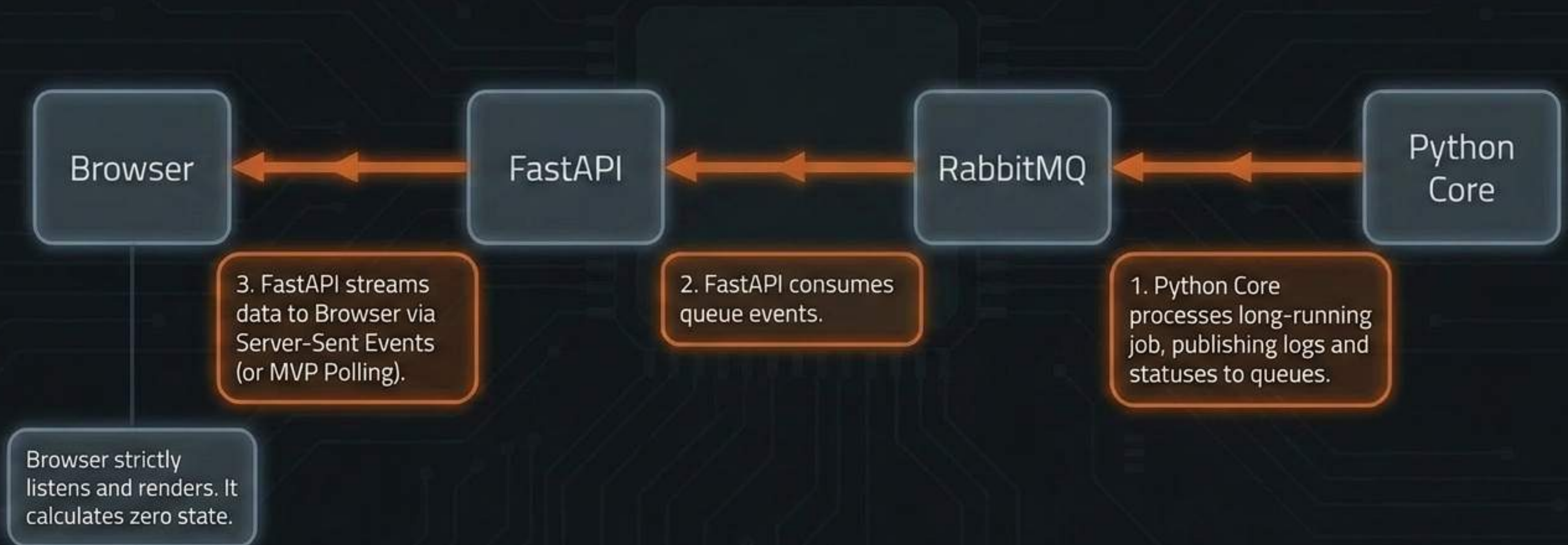


Decision: RabbitMQ bridges internal security zones. The architecture remains network-independent, eliminating brittle point-to-point HTTP links.

Mechanical Flow 1: Command Execution



Mechanical Flow 2: Runtime Telemetry

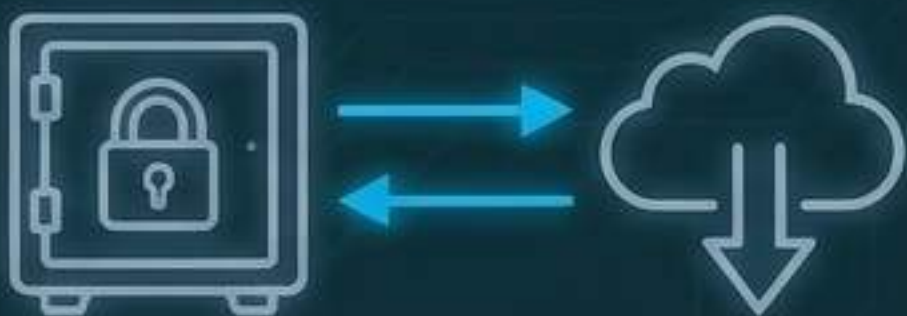


State Ownership & Boundaries

Rule: Never duplicate backend runtime state manually.

Server State (TanStack Query)

- Async jobs
- Streaming Logs
- Backend statuses
- Runtime telemetry



Client State (Zustand)

- Active UI tabs
- Selected entity panels
- Expanded menus



Mitigating Architecture Traps

If treated as standard CRUD...

Giant monolithic
UI components.

Duplicated API
fetch loops.

UI freezes waiting
for long runs.

Frontend attempts to
guess backend state.

Async Orchestration Approach...

Strict component modularity.

TanStack Query centralizes polling.

Decoupling via RabbitMQ.

Failure-resilient, partial-load UX.

Incremental Delivery Strategy



AI Development Guardrails

AI is an accelerator, not an architect.



AI Writes

UI scaffolding
Standard boilerplate
Material UI forms
TypeScript types



Humans Own

System architecture
Async lifecycles
Backend API contracts
State ownership

The Blueprint for Success



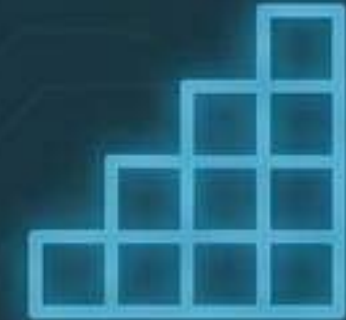
Protect the Core

The Python legacy backend remains untouched, safely shielded behind the RabbitMQ boundary.



Embrace Async

A frontend architecture built explicitly to handle long-running, asynchronous operational lifecycles.



Deliver Incrementally

A pragmatic, polling-first MVP ensures rapid deployment without premature overengineering.